SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA0308	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M Poojitha Reddy	**Reg. No.:**	192472352
Section / Batch:	2024	**Date:**	

Code of Conduct

I ***___M Poojitha Reddy - 192472352_____*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ____**M Poojitha Reddy_____**

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**

Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. **treatment**
2. **treatable**
3. **retreatment**
4. **treated**
5. **untreated**

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. **Identify words that produce incorrect morphological analyses.**
2. **Decompose the selected words into prefixes, roots, and suffixes.**
3. **Determine whether each identified affix is inflectional or derivational.**
4. **Explain the root cause of the incorrect morphological analysis.**
5. **Propose a corrected morphological-analysis strategy.**
6. **Compare the original and corrected analyses.**
7. **Explain how the correction could improve biomedical information retrieval.**

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. **replayed**
2. **unhappier**
3. **disconnected**
4. **players**
5. **restarting**
6. **unreadable**

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. **Identify words that produce incorrect morphological analyses.**
2. **Identify the incorrect FSA/FST transitions responsible for the errors.**
3. **Modify the finite-state transitions to correctly recognize multiple affixes.**
4. **Execute the corrected parser on the selected dataset.**
5. **Compare the parser output before and after correction.**
6. **Calculate the parsing accuracy before and after correction.**
7. **Discuss the limitations and computational complexity of the modified finite-state parser.**

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. **Identify the programming and preprocessing errors.**
2. **Explain why the output is incorrect.**

3. **Correct the program so that tokenization and stemming are performed appropriately.**
4. **Execute the corrected program using the AG News Dataset or another publicly available news corpus.**
5. **Compare:**
    - **Original text**
    - **Tokens**
    - **Stemmed tokens**
6. **Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.**
7. **Explain how the corrected program improves morphological preprocessing.**

Question 4 – Error Analysis of a Finite-State Morphological Parser
The following Python program is intended to identify plural nouns:

```
def parser(word):
if word.endswith("s"):
      return word[:-2], "Plural Noun"
else:
      return word, "Singular"

words = [
   "cars",
   "boxes",
   "cities",
   "children",
   "books"
]

for w in words:
   print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.
Tasks

1. **Identify all logical errors in the program.**
2. **Explain why the parser fails for different plural formation rules.**
3. **Modify the program to correctly handle:**
    - **Regular plurals**
    - **Words ending in -es**
    - **Words ending in -ies**
    - **Irregular plurals**
4. **Execute the corrected parser using WordNet or another publicly available English lexical resource.**
5. **Compare the original and corrected outputs.**
6. **Identify the limitations of the rule-based finite-state approach.**
7. **Explain how the corrected parser improves morphological analysis.**

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction
The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
        "connected connection connecting connectivity",
        "studies studied studying study",
        "organize organized organizer organization"
]

vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(documents)

features = [
        stemmer.stem(word)
        for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. **Identify the preprocessing error in the program.**
2. **Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.**
3. **Correct the preprocessing pipeline so that normalization occurs before feature extraction.**
4. **Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.**
5. **Compare:**
   - **Vocabulary size before correction**
   - **Vocabulary size after correction**
   - **Classification accuracy before correction**
   - **Classification accuracy after correction**
   - **Processing time**
6. **Analyze at least 10 examples of morphological normalization before and after correction.**
7. **Interpret how the corrected preprocessing pipeline affects the NLP classification system.**

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts

Clarity & Presentation	**10%**	**Presents analysis clearly, logically, and systematically**	**Generally clear with minor issues**	**Lacks clarity or proper structure**	**Poor presentation and difficult to understand**

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	4	4	3	4	19	97
2	4	4	4	3	4	19	
3	4	4	4	4	4	20	
4	4	4	3	4	4	19	
5	4	4	4	4	4	20	

Faculty In-charge	Course Coordinator	Program Director
T Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Error Analysis of Derivational Morphology

1. Correct morphological decomposition

Word	Decomposition	Affix type
treatment	treat + ment	`-ment` = Derivational
treatable	treat + able	`-able` = Derivational
retreatment	re + treat + ment	`re-`, `-ment` = Derivational
treated	treat + ed	`-ed` = Inflectional
untreated	un + treat + ed	`un-` = Derivational, `-ed` = Inflectional

2. Error in the system

The system incorrectly performs blind prefix/suffix removal. It treats all affixes as removable without identifying whether they are derivational or inflectional.

For example:

```
untreated → treat
```

loses the important information **un- (negation)**.

3. Correct strategy

The analyzer should identify:

```
Prefix → Root → Suffix
```

and classify every affix.

Example:

```
untreated
= un- + treat + -ed
= derivational + root + inflectional
```

4. Original vs corrected

```
Original:   untreated → treat
Corrected:  untreated → un + treat + ed
```

5. Effect on information retrieval

The corrected method preserves morphological meaning. Thus, terms such as **treated** and **untreated** are not incorrectly treated as identical.

Conclusion: Morphological analysis should not simply remove affixes; it must preserve their grammatical and semantic information.

Q2. Error Analysis of Finite-State Morphological Parsing

1. Correct analyses

Word	Correct FSA/FST analysis
replayed	re + play + ed
unhappier	un + happy + er
disconnected	dis + connect + ed
players	play + er + s
restarting	re + start + ing
unreadable	un + read + able

2. Error

The original parser recognizes only a **single affix**. Therefore, it fails when multiple prefixes/suffixes occur. For example:

```
replayed = re + play + ed
```

requires both a prefix transition and a suffix transition.

3. Correct transition structure

Instead of:

```
START → PREFIX → ROOT → SUFFIX → END
```

the parser should allow repeated affix transitions:

```
START → PREFIX* → ROOT → SUFFIX* → END
```

where * means **zero or more occurrences**.

4. Example

```
replayed

 START
   ↓
  re-
   ↓
 play
   ↓
 -ed
   ↓
 END
```

Analysis:

```
re-  → derivational
play → root
-ed  → inflectional
```

5. Accuracy

For the six manually selected test words:

$$\textbf{Accuracy=Total words/Correct analyses×100}$$

If the original parser correctly handles 2/6:

$$\textbf{2/6 * 100 = 33.33\%}$$

After correction:

6/6 * 100 = 100%

6. Limitation

- The rule-based FSA/FST may fail for irregular words, spelling changes, unknown words and ambiguous analyses.
- Conclusion: Allowing multiple affix transitions makes the parser capable of handling complex morphological structures.

Q3. Porter Stemmer — Identify and Correct the Error

Given error

```
data["Processed"] = data["Text"].apply(ps.stem)
```

1. Error

`PorterStemmer.stem()` expects **one word**, but the program passes an entire sentence/document.

Therefore, tokenization is missing.

2. Correct program

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()
data = pd.read_csv("news.csv")
```

```
def process(text):
    tokens = re.findall(r'\b[a-zA-Z]+\b', str(text).lower())
    stems = [ps.stem(word) for word in tokens]
    return tokens, stems

data[["Tokens", "Stemmed"]] = data["Text"].apply(
    lambda x: pd.Series(process(x))
)

print(data[["Text", "Tokens", "Stemmed"]].head())
```

3. Example output

For:

```
Researchers conducted studies.
```

Tokens:

```
researchers, conducted, studies
```

Stemmed tokens:

```
research, conduct, studi
```

4. Twenty important stemming cases

Word	Stem	Type
studies	studi	Inflectional
studied	studi	Inflectional
studying	studi	Inflectional
connected	connect	Inflectional
connecting	connect	Inflectional
connections	connect	Inflectional
treatments	treatment	Inflectional
researchers	research	Inflectional
players	player	Inflectional
games	game	Inflectional
organization	organ	Derivational
organize	organ	Derivational

Word	Stem	Type
happiness	happi	Derivational
happily	happili	Derivational
readable	readabl	Derivational
kindness	kind	Derivational
national	nation	Derivational
nationalize	nation	Derivational
agreement	agre	Derivational
development	develop	Derivational

Note: Porter stemming is heuristic, so some outputs such as `organization → organ` are linguistically aggressive.

5. Improvement

Correct preprocessing:

```
Document → Tokenization → Stemming
```

reduces vocabulary size and sparsity and improves matching in information retrieval.

Conclusion: Stemming must be performed after tokenization and before feature extraction.

Q4. Finite-State Morphological Parser — Plural Nouns

1. Errors

The main error is:

```
word[:-2]
```

It removes **two characters from every word ending in `s`**.

For example:

```
cars → ca      ❌
books → boo    ❌
cities → cit   ❌
```

It also cannot handle irregular plurals such as:

```
children → child
```

2. Correct program

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "feet": "foot",
        "teeth": "tooth"
    }

    if word in irregular:
        return irregular[word], "Plural Noun"

    if word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun"

    if word.endswith("es"):
        return word[:-2], "Plural Noun"

    if word.endswith("s"):
        return word[:-1], "Plural Noun"

    return word, "Singular Noun"


words = ["cars", "boxes", "cities", "children", "books"]

for w in words:
    print(w, "->", parser(w))
```

3. Correct output

```
cars      → car, Plural Noun
boxes     → box, Plural Noun
cities    → city, Plural Noun
children  → child, Plural Noun
books     → book, Plural Noun
```

4. Rules used

```
Regular:  books  → book
-es:      boxes  → box
-ies:     cities → city
Irregular: children → child
```

5. Original vs corrected

Word	Original	Correct
cars	ca	car
boxes	box	box
cities	cit	city
children	childre	child
books	boo	book

6. Limitation

A rule-based parser cannot handle every English exception. It needs a **lexicon/dictionary** for irregular forms and may also fail with ambiguous or unknown words.

Conclusion: Separate morphological rules plus an irregular-word dictionary provide more accurate plural analysis.